

Módulo 1: Fundamentos y Mecánica de JavaScript

1.1 ¿Qué es JavaScript realmente?

JavaScript es un lenguaje de programación de alto nivel, interpretado y multiparadigma. A diferencia de lenguajes compilados (como C++), JavaScript se ejecuta generalmente en un motor (como V8 en Chrome o Node.js) que traduce el código a lenguaje máquina en tiempo real (compilación Just-In-Time).

El Entorno de Ejecución

Navegador: Permite interactuar con el DOM (páginas web).

node.js: Permite usar JavaScript en servidores, accediendo al sistema de archivos y bases de datos

1.2 Variables y Constantes: El manejo de memoria

En JavaScript moderno, la forma en que declaramos variables define el alcance (scope) y la mutabilidad de los datos.

const: Se utiliza para valores que no cambiarán su referencia. Es la opción por defecto para un código limpio.

let: Se utiliza para valores que necesitan ser reasignados. Tiene "alcance de bloque" (solo vive dentro de las llaves {} donde se creó).

var: (Obsoleto/No recomendado). Tiene alcance de función y sufre de hoisting (elevación), lo que suele generar errores lógicos

1.3 Tipos de Datos (Primitivos y Objetos)

JavaScript es dinámicamente tipado, lo que significa que una variable puede cambiar de tipo, pero los datos en sí tienen categorías claras:

Primitivos (Inmutables)

1. **Number:** Representa tanto integrales como decimales (ej. 42 o 3.14).
2. **String:** Cadenas de texto. Se recomienda el uso de backticks (`) para plantillas literales.
3. **Boolean:** Solo dos valores: true o false.
4. **Null:** Representa la ausencia intencional de valor.
5. **Undefined:** Valor de una variable que ha sido declarada pero no inicializada.
6. **Symbol:** Identificadores únicos para propiedades de objetos.
7. **BigInt:** Para números enteros de precisión arbitraria.

Especiales

Object: Una colección de pares clave-valor (incluye arrays y funciones).

1.4 Operadores Esenciales

Para manipular los datos, necesitamos herramientas de operación:

- Aritméticos: +, -, *, /, % (módulo) y ** (exponenciación).
- Asignación Compuesta: +=, -=, * =.
- Comparación Estricta:
 - === (Igualdad estricta): Compara valor y tipo (Siempre usa este).
 - == (Igualdad débil): Realiza conversión de tipos automática (Evítalo).
- Lógicos: && (AND), || (OR), ! (NOT).

1.5 Estructuras de Control de Flujo

Determinan qué camino toma el código basado en condiciones.

Condicionales

- if / else if / else: La lógica estándar de decisión.
- Operador Ternario: Una forma elegante de escribir un if/else en una sola línea: `condicion ? expresion_si_true : expresion_si_false;`
- switch: Ideal para evaluar múltiples casos específicos de una sola variable.

Bucles (Loops)

- for: Cuando sabemos cuántas veces iterar.
- while: Mientras una condición sea verdadera.
- for...of: La forma moderna y legible de recorrer elementos de un array.

1.6 Funciones: Los bloques de construcción

Una función es un conjunto de instrucciones reutilizables.

Declaración tradicional:

```
function saludar(nombre) {  
    return `Hola, ${nombre}`;  
}
```

Funciones de Flecha (Arrow Functions):
Introducidas en ES6, son más concisas y no tienen su propio contexto de this.

```
const saludar = (nombre) => `Hola, ${nombre}`;
```